
FPGA Tick-to-Trade

Part 2 — RTL Design & Verification

Abhishek · June 2026 · TESTS=81 PASS=81 FAIL=0

SystemVerilog RTL · cocotb · QuestaSim 2021.2 · Python golden model

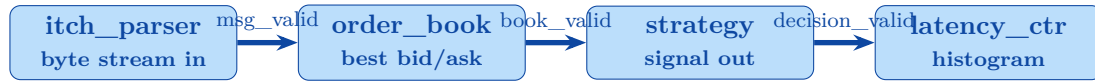
Contents

1	RTL Pipeline Overview	1
2	Module 1: <code>itch_parser.sv</code>	1
2.1	What the Parser Does	1
2.2	Shift Register Accumulation	1
2.3	ITCH 5.0 Message Byte Layouts	1
2.4	Parser State Machine	2
2.5	Critical Bug: Non-Blocking Assignment Timing	2
2.5.1	What Are Non-Blocking Assignments?	2
2.5.2	The Bug	2
2.5.3	The Fix: Combinational <code>_nxt</code> Wires	3
2.6	Tests that Failed and Why	3
3	Module 2: <code>order_book_top.sv</code>	4
3.1	Why Order Book Reconstruction?	4
3.2	Milestone 1 Data Structure: Direct-Mapped Register File	4
3.3	Message Processing	4
3.4	Known Bug Fixed: <code>table</code> is a Reserved Keyword	5
4	Module 2b: <code>order_book_m2.sv</code> — The RESCAN FSM	5
4.1	The Stale-Best Problem	5
4.2	Design: A Two-State FSM with a Single-Pass Rescan	5
4.3	The Single-Pass Best-of-N Algorithm	5

4.4	The Off-by-One Trap: Why the Scan Runs 257 Cycles	6
4.5	The Cost, and the Optimisations Now Applied	6
4.6	Verification — tb_order_book_m2.py (28/28 PASS)	6
5	Module 3: strategy_imbalance.sv	7
5.1	The Book Imbalance Signal	7
5.2	Fixed-Point Cross-Multiply Implementation	7
6	Module 4: latency_counter.sv	8
6.1	Why Hardware Latency Measurement?	8
6.2	Ideal Histogram: Single Spike	8
7	Module 5: tick_to_trade_top.sv	8
8	Verification Framework	9
8.1	Why Golden Model Comparison?	9
8.2	The cocotb-Side NBA Trap: Reading Registered Outputs	9
8.3	Test Results — 81/81 PASS	10
8.4	Issues Log	10
9	Milestone 2 Feature Additions	10
9.1	Full ITCH message set (8 types)	11
9.2	Depth-weighted strategy and risk controls	11
9.3	AXI-Lite latency readout	11
10	What's Next	11

1 RTL Pipeline Overview

The five RTL modules form a strictly linear pipeline. Data flows in one direction only; every module is clock-synchronous at the same frequency. A valid/ready handshake between the parser and the order book provides back-pressure, so a message arriving while the book is busy (during a rescan) is held, never dropped.



Interface contract: Every module presents data using a valid-pulse protocol. A signal named `*_valid` is asserted for exactly **one clock cycle** carrying the data in that same cycle. The downstream module must be ready to accept it (no stalling).

2 Module 1: itch_parser.sv

2.1 What the Parser Does

The parser is an **AXI-Stream slave**. It ingests one byte per clock cycle, accumulates bytes into field shift registers, and emits a one-cycle `m_valid` pulse on the cycle of the last byte (`tlast`). The outputs — `msg_type`, `timestamp`, `order_ref`, `side`, `shares`, `price` — are stable at that cycle and held until the next `m_valid`.

2.2 Shift Register Accumulation

Each multi-byte field is accumulated using a **left-shifting shift register**. For a 64-bit `order_ref` field spanning bytes 11–18:

Listing 1: Left-shifting shift register: 8 bytes into 64 bits

```

1 // ref_sr holds the accumulated value so far
2 // On each byte that belongs to order_ref:
3 ref_sr <= {ref_sr[55:0], s_axis_tdata}; // shift left, insert new byte

```

After byte 18 the register contains the correct big-endian 64-bit value. This requires **zero multiply, zero divide** — just concatenation (wiring).

2.3 ITCH 5.0 Message Byte Layouts

Each message type is a fixed-length binary record. Byte 0 is always the message type character (ASCII). The parser routes subsequent bytes using `case(byte_cnt)`.

Add Order (A) — 36 bytes

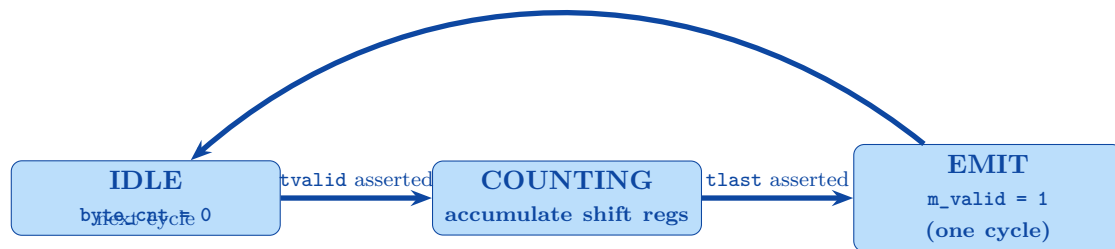
Bytes	Field	Notes
0	<code>msg_type</code>	0x41 = 'A'
1–2	<code>stock_locate</code>	2B big-endian
3–4	<code>tracking_number</code>	2B
5–10	<code>timestamp</code>	6B nanoseconds
11–18	<code>order_ref</code>	8B unique ID
19	<code>side</code>	'B'=bid, 'S'=ask
20–23	<code>shares</code>	4B
24–31	<code>stock</code>	8B symbol (ignored)
32–35	<code>price</code>	4B fixed-point ×10,000

Cancel (X) — 23 bytes and **Delete (D) — 19 bytes**

Bytes	Field	Cancel (X)	Delete (D)
0	msg_type	0x58	0x44
1–4	locate/tracking	same	same
5–10	timestamp	same	same
11–18	order_ref	bytes 11–18, tlast=byte 18	bytes 11–18, tlast=byte 18
19–22	cancelled shares	4B (Cancel only)	—

Key observation: For Cancel and Delete, the last parsed field (`order_ref`) ends on the same byte as `tlast`. This is the root cause of the NBA timing bug described in Section 2.5.

2.4 Parser State Machine



In COUNTING state, `byte_cnt` increments on each `tvalid` cycle. A `case(byte_cnt)` routes each byte to its field shift register using `always_comb` (`_nxt` wires). On `tlast`: outputs latch the `_nxt` values.

2.5 Critical Bug: Non-Blocking Assignment Timing

2.5.1 What Are Non-Blocking Assignments?

In SystemVerilog, `always_ff` blocks use the non-blocking assignment operator (`<=`). The IEEE 1800 standard defines two phases per clock edge:

1. **Active region:** Right-hand sides of all `<=` assignments are evaluated using the *current* (pre-edge) values of all signals.
2. **NBA region:** The computed values are written to the registers.

This means: if `A <= B + 1` and `C <= A`, both use the **old** value of `A`. This is correct and intentional for pipelined RTL — but it creates a subtle trap when capturing a signal on the same cycle it is being updated.

2.5.2 The Bug

When the last byte of `order_ref` (byte 18) arrives on the same cycle as `tlast`, the `always_ff` block evaluates `order_ref <= ref_sr` using the *pre-edge* value of `ref_sr` (bytes 11–17 only). Byte 18 was just presented to the shift register on that edge — it will be in `ref_sr` only *after* the NBA update, one cycle later. The output `order_ref` is therefore missing the last byte.

Signal	At tlast rising edge	Effect
<code>ref_sr</code>	holds bytes 11–17 (pre-edge)	NBA not yet applied
<code>order_ref</code> (BUG)	captures <code>ref_sr</code> = bytes 11–17	missing byte 18!
<code>order_ref</code> (FIX)	captures <code>ref_nxt</code> = bytes 11–18	correct

2.5.3 The Fix: Combinational `_nxt` Wires

Add an `always_comb` block that computes the *next* value of each shift register including the *current* byte. The `always_ff` block then uses these `_nxt` signals for both the register update and the output latch:

Listing 2: Combinational `_nxt` fix — the core pattern in `itch_parser.sv`

```

1 // COMBINATIONAL: compute next-state shift register values
2 // This sees s_axis_tdata (the CURRENT byte) right now.
3 always_comb begin
4     ref_nxt = ref_sr;           // default: hold
5     if (tvalid && tready) begin
6         case (byte_cnt)
7             6'd11, 6'd12, 6'd13, 6'd14,
8             6'd15, 6'd16, 6'd17, 6'd18:
9                 ref_nxt = {ref_sr[55:0], s_axis_tdata}; // include current
10                        byte
11         endcase
12     end
13 end
14 // SEQUENTIAL: on clock edge, use _nxt (not _sr)
15 always_ff @(posedge clk or negedge rst_n) begin
16     if (!rst_n) begin
17         ref_sr    <= '0;
18         order_ref <= '0;
19         m_valid   <= 1'b0;
20     end else begin
21         ref_sr    <= ref_nxt;           // normal pipeline update
22         m_valid   <= 1'b0;             // default: deassert
23         if (s_axis_tlast) begin
24             order_ref <= ref_nxt;       // FIX: _nxt includes the current byte
25             m_valid   <= 1'b1;         // one-cycle pulse
26         end
27     end
28 end

```

The `always_comb` block is combinational logic — it evaluates instantaneously. Because it includes `s_axis_tdata` on the right-hand side, `ref_nxt` already contains the correct shifted value before the clock edge arrives. The `always_ff` block then latches `ref_nxt`, not the stale `ref_sr`.

2.6 Tests that Failed and Why

Test	Msg	Last field ends at	= tlast?	Pre-fix
test_add_order	Add (36B)	price byte 35	YES	FAIL
test_sell_add	Add (36B)	side byte 19	no	PASS
test_cancel	Cancel (23B)	shares byte 22	YES	FAIL
test_delete	Delete (19B)	order_ref byte 18	YES	FAIL
test_execute	Execute (31B)	shares byte 22	no (last=30)	PASS
test_back_to_back	mixed	—	—	FAIL

Tests fail when the *last byte of the last captured field* coincides with `tlast`. The `_nxt` fix resolves all six cases.

3 Module 2: order_book_top.sv

3.1 Why Order Book Reconstruction?

A raw ITCH feed does not tell you the current price. It only tells you what *happened*: “order 1234 was added at \$150.01 for 500 shares.” To know the best available bid and ask at any moment, you must maintain a **limit order book**: a data structure that tracks all live orders and exposes the best (highest bid, lowest ask) at all times.

3.2 Milestone 1 Data Structure: Direct-Mapped Register File

The order book is implemented as a **256-entry register file**, directly indexed by the low 8 bits of the 64-bit `order_ref` number.

Field	Width / Notes
<code>valid</code>	1 bit — 1 = live order in this slot
<code>order_ref</code>	64 bits — full reference number
<code>side</code>	1 bit — 0=BID, 1=ASK
<code>price</code>	32 bits — fixed-point $\times 10,000$
<code>shares</code>	32 bits — remaining shares
Index	<code>order_ref[7:0]</code> — direct map, O(1) lookup

Why direct-mapped? O(1) lookup with no comparator chain. No sorting, no priority encoding required. `order_book_m2` extends this into a **multi-level** structure that tracks the top 4 price levels per side (price + aggregated size), while keeping the Add path O(1) (see Section on the RESCAN FSM).

Struct definition:

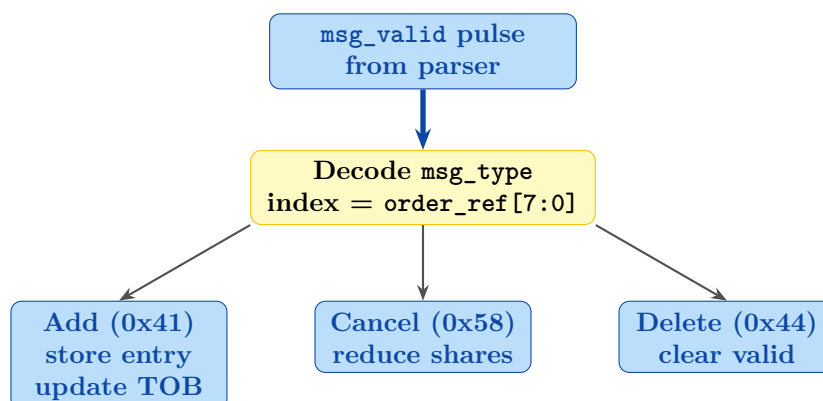
Listing 3: Order entry typedef in `order_book_top.sv`

```

1 typedef struct packed {
2     logic        valid;           // 1 = this slot has a live order
3     logic [63:0] order_ref;       // full 64-bit order reference number
4     logic        side;           // 0=BID, 1=ASK
5     logic [31:0] price;          // price * 10,000 (fixed-point)
6     logic [31:0] shares;         // remaining shares
7 } order_entry_t;
8
9 order_entry_t entries [0:ORDER_DEPTH-1]; // 256 entries

```

3.3 Message Processing



M1 limitation: best bid/ask is NOT re-scanned after cancel/delete. This is the central problem solved in Milestone 2 (Section 4).

3.4 Known Bug Fixed: table is a Reserved Keyword

SystemVerilog reserves the keyword `table` for UDP (User-Defined Primitive) truth tables. Using it as a variable name causes a parse error:

Original (broken)	Fixed
<code>order_entry_t table [...];</code>	<code>order_entry_t entries [...];</code>

4 Module 2b: order_book_m2.sv — The RESCAN FSM

4.1 The Stale-Best Problem

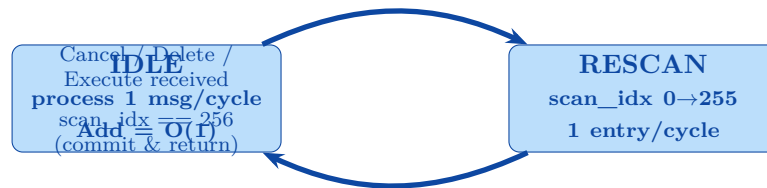
Milestone 1's order book is correct *only* for Add messages. When an order is cancelled, deleted, or partially executed, M1 updates the stored entry but **never re-examines the best bid/ask**. Consider this sequence:

Step	Message	M1 best_bid	Correct best_bid
1	Add ref=1 bid \$100 × 50	\$100 × 50	\$100 × 50
2	Add ref=2 bid \$90 × 30	\$100 × 50	\$100 × 50
3	Delete ref=1 (the \$100 bid)	\$100 × 50 (<i>stale!</i>)	\$90 × 30

After step 3 the \$100 order is gone, yet M1 still reports it as the best bid. A strategy reading this would trade on a price that no longer exists — in HFT terms, a **phantom quote**. M2 fixes this.

4.2 Design: A Two-State FSM with a Single-Pass Rescan

The fix is conceptually simple: after any Cancel/Delete/Execute, re-scan *all* 256 entries and recompute the best bid and ask from scratch. The challenge is doing it in hardware without a comparator tree across 256 entries (which would never meet timing at 200 MHz). The solution is to **spread the scan over time**: one entry per clock cycle, using a small FSM.



Add path is untouched — it stays $O(1)$ in the IDLE state, exactly as in M1. Only the destructive messages (C/D/E) pay the rescan cost. This matters: Add is by far the most common message in a real feed, so the common case stays single-cycle.

4.3 The Single-Pass Best-of-N Algorithm

During RESCAN, the FSM keeps running accumulators (`scan_bid_p/s`, `scan_ask_p/s`). At each entry it applies the classic *find-max-in-one-pass* rule, extended to also **aggregate size at the winning price**:

Listing 4: Per-entry rescan logic — bid side (ask is the mirror with $<$)

```

1 if (entries[scan_idx].valid && !entries[scan_idx].side) begin // a bid
2   if (!scan_bid_seen || entries[scan_idx].price > scan_bid_p) begin
3     // strictly better price: start a fresh best level
4     scan_bid_p <= entries[scan_idx].price;
  
```

```

5      scan_bid_s      <= entries[scan_idx].shares;
6      scan_bid_seen <= 1'b1;
7  end else if (entries[scan_idx].price == scan_bid_p) begin
8      // same price as current best: add this order's shares to the level
9      scan_bid_s <= scan_bid_s + entries[scan_idx].shares;
10 end
11 // worse price: ignore
12 end

```

For the ask side the only change is < instead of > (lowest ask wins). When `scan_idx` reaches 256, the accumulated values are committed to the output registers and the FSM returns to IDLE.

4.4 The Off-by-One Trap: Why the Scan Runs 257 Cycles

A 256-entry table needs **257** cycles, not 256. This is the same NBA subtlety from Section 2.5, now on the read side. When the FSM examines the last entry (`scan_idx == 255`), the accumulator update is a non-blocking assignment — it does not take effect until the *next* clock edge. If we committed on the cycle `scan_idx == 255`, we would read the accumulators *before* entry 255 was folded in, silently dropping the last order.

Cycle	Action	State of accumulators
0–254	scan entries 0–254	folding each entry in
255	scan entry 255	entry 255's NBA <i>scheduled</i> , not yet visible
256	<code>scan_idx == 256: COMMIT</code>	entry 255's NBA now settled — read safely

The extra commit cycle is why `scan_idx` is declared `[ADDR_W:0]` (9 bits, reaching 256) rather than `[ADDR_W-1:0]` (8 bits, wrapping at 255). **The same NBA rule bites twice in this project** — once writing (parser), once reading (rescan commit).

4.5 The Cost, and the Optimisations Now Applied

A worst-case RESCAN takes $257 \text{ cycles} \times 5 \text{ ns} = 1.285 \mu\text{s}$ at 200 MHz. Two optimisations (both now implemented) keep this off the critical path in the common case:

Concern	Original M2	Now (implemented)
Best after C/D/E	correct (rescan)	correct
Add latency	1 cycle	1 cycle
Msgs during rescan	dropped	held (<code>msg_ready</code> backpressure)
Rescan trigger	every C/D/E	only if a <i>displayed</i> level is touched

Back-pressure: the book asserts `msg_ready` only in IDLE; the parser holds `m_valid` (and stalls its byte input) until the book accepts, so nothing is dropped during a rescan. **Selective rescan:** a Cancel/Delete/Execute on an order priced *beyond the tracked depth* cannot change any shown level, so it skips the rescan entirely and only updates the full-depth order table. Correctness first (obvious full rescan), then these two safe optimisations on top.

4.6 Verification — `tb_order_book_m2.py` (28/28 PASS)

The M2 testbench targets exactly the behaviours M1 got wrong. The **headline test** drives the stale-best sequence from the top of this section and asserts the corrected result:

Listing 5: The test that proves M2 fixes the phantom quote

```

1  async def test_delete_best_bid_reveals_second(dut):
2      await drive_msg(dut, ADD, order_ref=1, side=BID, shares=50, price=1000
        _0000)

```



```

3      await drive_msg(dut, ADD, order_ref=2, side=BID, shares=30, price=900
        _0000)
4      check(dut, bid_p=1000_0000, bid_s=50, tag="before_delete")
5
6      await drive_msg(dut, DELETE, order_ref=1, side=BID, shares=0, price=0)
7      await wait_rescan(dut) # wait out the 257-cycle FSM
8      check(dut, bid_p=900_0000, bid_s=30, tag="after_delete_best_bid") # M1
        fails here

```

Test	What it proves	Status
delete_best_bid_reveals_second	deleting the best bid promotes the 2nd-best	PASS
delete_non_best_bid_unchanged	deleting a lower bid leaves best intact	PASS
cancel_partial_reduces_size	partial cancel updates size after rescan	PASS
cancel_full_empties_book	removing last order drops book_valid to 0	PASS
execute_reduces_size	execute decrements remaining shares	PASS
two_asks_delete_best_reveals_higher	ask side mirror of the bid test	PASS
aggregate_then_partial_cancel	size aggregation survives a partial cancel	PASS
+ basic Add/level, replace, rescan-skip, AXI tests		TOTAL 28/28

Hash-collision gotcha: the table is indexed by `order_ref[7:0]`, so `order_ref=1` and `order_ref=257` map to the *same* slot. Early test drafts used 257 as a “second order” and silently overwrote the first. Tests now use small, distinct refs (1, 2, 10, 11) that occupy different slots.

5 Module 3: strategy_imbalance sv

5.1 The Book Imbalance Signal

Book imbalance is a well-known short-term price predictor in market microstructure. The observable bid-ask imbalance is a proxy signal for informed order flow:

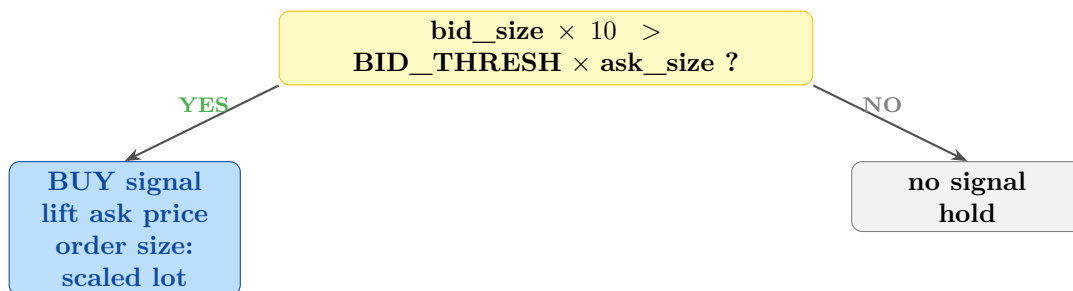
$$\text{Imbalance} = \frac{\text{bid_size}}{\text{bid_size} + \text{ask_size}} \in [0, 1]$$

A value near 1 means bid is dominant (buyers outnumber sellers) $\Rightarrow$ price likely to rise $\Rightarrow$ **BUY** signal. A value near 0 means ask is dominant $\Rightarrow$ **SELL** signal.

The core test below uses the top-of-book sizes for clarity; the shipped strategy generalises this to a *depth-weighted* volume over the top 4 levels and adds a spread guard, a post-trade cooldown and imbalance-scaled lot sizing (see Section 9, Milestone 2 Feature Additions).

5.2 Fixed-Point Cross-Multiply Implementation

Division is implemented as cross-multiplication to avoid any floating-point unit or multi-cycle divider:



Default parameter: `BID_THRESH = 15` corresponds to $\frac{15}{10} = 1.5\times$. The bid must be at least $1.5\times$ larger than the ask (by share count) to trigger a BUY. One DSP block per side. One clock cycle, no division.

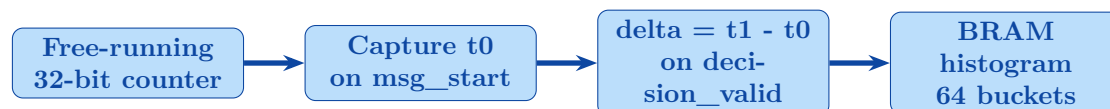
6 Module 4: latency_counter.sv

6.1 Why Hardware Latency Measurement?

Software timing using `clock_gettime()` has a resolution of $\sim 1\ \mu\text{s}$ and adds measurement jitter. A hardware counter with a 5 ns clock period can measure latency to **single-cycle precision**.

The counter architecture:

- A free-running 32-bit counter increments every clock cycle
- `t0` is captured when the AXI-Stream `tvalid` rises
- `t1 - t0` (the *delta*) is computed when `decision_valid` fires
- The delta is binned into a 64-bucket BRAM histogram (bucket = delta / 4 cycles)



6.2 Ideal Histogram: Single Spike

On an FPGA, every identical message takes the same number of cycles. The ideal histogram has a **single bin** with count = number of messages:

Latency bucket	Count	Note
0–16 ns (buckets 0–3)	0	below pipeline latency
20 ns (bucket 5)	100	all 100 messages land here — single spike
24–48 ns (buckets 6–12)	0	no jitter above baseline

Every message takes exactly 20 ns (5 buckets at 200 MHz). A CPU would show a distribution spanning 3–4 orders of magnitude. This single spike is the visual proof of hardware determinism.

7 Module 5: tick_to_trade_top.sv

The top-level wrapper instantiates all four modules and wires the inter-module signals. It packages the trading decision into a single 72-bit bus:

Bits	Signal	Description
[71:65]	reserved (7'b0)	padding to 72-bit boundary
[64]	<code>action</code>	0 = BUY, 1 = SELL
[63:32]	<code>order_price</code>	limit price (fixed-point $\times 10,000$)
[31:0]	<code>order_size</code>	number of shares (default 100)

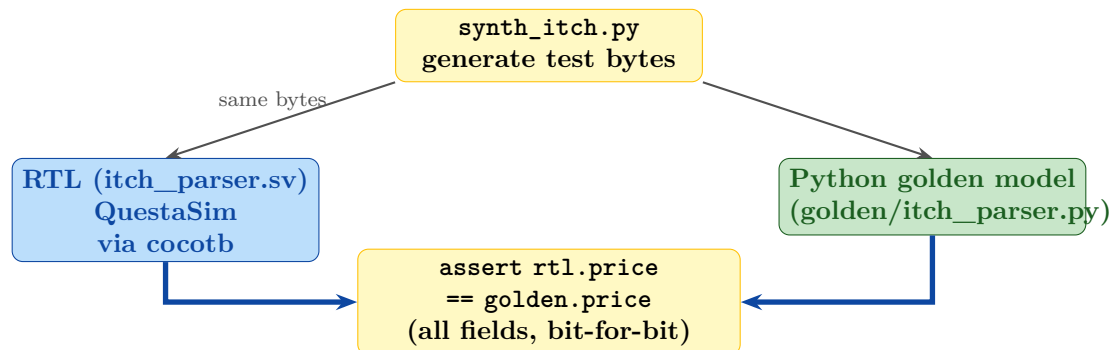
This bus is read by the EC2 host via PCIe DMA, decoded, and passed to `ibkr_bridge.py`.

8 Verification Framework

8.1 Why Golden Model Comparison?

A “golden model” is a functionally correct reference implementation in Python. Instead of writing expected values by hand (error-prone for complex protocols), the testbench:

1. Generates the same test vector for both RTL and the golden model
2. Runs both in parallel
3. Asserts that every output field matches bit-for-bit



8.2 The cocotb-Side NBA Trap: Reading Registered Outputs

Section 2.5 described the NBA rule *inside* the RTL. The same rule bites in the **Python testbench**, and it is the single most common cocotb mistake. After `await RisingEdge(dut.clk)`, cocotb resumes in the **Active region** — *before* Questa has committed that edge’s non-blocking assignments. So a registered output you expect to have just changed still shows its *old* value.

Symptom	Cause
Output reads 0 right after driving an input	<code>always_ff</code> fired, but its <code><=</code> has not committed when cocotb reads in the Active region of the same edge

The fix is to wait one *additional* `RisingEdge` after the edge that triggers the update, letting the NBA settle into the Active region of the following cycle. Every driver helper in this project bakes this in:

Listing 6: The two-edge `drive_msg` pattern (used in every order-book test)

```

1 async def drive_msg(dut, msg_type, order_ref, side, shares, price):
2     dut.msg_valid.value = 1
3     dut.msg_type.value = msg_type
4     # ... drive the rest of the inputs ...
5     await RisingEdge(dut.clk) # always_ff fires; NBAs are SCHEDULED
6     dut.msg_valid.value = 0
7     await RisingEdge(dut.clk) # NBAs from the previous edge now COMMITTED
    -> safe to read
  
```

The integration test hit a two-deep version of this: the strategy sets `decision_valid` at edge T , the latency counter reacts at $T+1$, so `last_latency_cycles` is only valid at $T+2$. The fix is to capture `m_axis_tdata` immediately on seeing `tvalid`, then `await` one more edge before reading the latency. **Rule of thumb:** one extra `RisingEdge` per pipeline register between the input you drove and the output you read.

8.3 Test Results — 81/81 PASS

The suite spans six testbenches across both order-book generations, every output diffed bit-for-bit against the Python golden model; all RTL is `verilator -Wall` lint-clean:

Testbench	DUT	Result	Focus
tb_itch_parser.py	itch_parser.sv	11/11	8 msg types, back-to-back, back-pressure hold
tb_order_book.py	order_book_top.sv	11/11	M1 top-of-book on Add
tb_order_book_m2.py	order_book_m2.sv	28/28	multi-level, RESCAN, replace, rescan-skip
tb_strategy_imbalance.py	strategy_imbalance.sv	21/21	spread guard, cooldown, lot tiers, depth weight
tb_latency_counter.py	latency_counter.sv	5/5	histogram, saturation, AXI-Lite read, clear
tb_tick_to_trade.py	tick_to_trade_top.sv	5/5	end-to-end 195 ns, back-pressure no-drop
TESTS=81 PASS=81 FAIL=0 81/81			

Headline number: the integration test measures a hardware-counted tick-to-trade latency of **39 cycles = 195 ns** at 200 MHz for an Add Order — 36 parser cycles + 1 book + 2 strategy gate.

8.4 Issues Log

Issue	Root Cause	Fix Applied
table syntax error	Reserved SV keyword (UDP truth tables)	Renamed array to <code>entries</code>
timescale error in Questa	Questa 2021.2 + <code>-mfcu:</code> per-file timescale mis-parsed	Removed from all RTL; added <code>-timescale 1ns/1ps</code> to Makefile
Wrong price / shares / order_ref	NBA timing: last field byte and <code>tlast</code> same cycle	Combinational <code>_nxt</code> wires; outputs use <code>_nxt</code> at <code>tlast</code>
Back-to-back test timeout	<code>collect_output</code> missed the one-cycle <code>m_valid</code> pulse	Parallel <code>monitor()</code> coroutine checks immediately
<code>units=</code> deprecation	cocotb 2.x renamed <code>units=</code> to <code>unit=</code> in <code>Clock()</code>	Updated all <code>Clock()</code> calls
Output reads stale 0 in testbench	cocotb resumes in Active region before NBA commits (Sec. 8.2)	Extra <code>await RisingEdge</code> per pipeline register before reading
M2 rescan dropped last order	Committed at <code>scan_idx==255;</code> entry 255's NBA not yet settled	Run 257 cycles; commit at <code>scan_idx==256</code> ([ADDR_W:0] width)
M2 test second order vanished	<code>order_ref=1</code> and 257 hash to same slot (<code>ref[7:0]</code>)	Use distinct low refs (1, 2, 10, 11) in tests
<code>import cocotb.test</code> fails	cocotb 2.x: <code>@cocotb.test()</code> lives in the cocotb namespace	Removed the bogus <code>import cocotb.test</code> line

9 Milestone 2 Feature Additions

Beyond the parser, M1/M2 books and basic strategy described above, the following were added and verified in this milestone.

9.1 Full ITCH message set (8 types)

The parser decodes eight ITCH 5.0 message types. Add-with-MPID (F) reuses the Add field layout plus a trailing attribution; Execute-with-Price (C) reuses Execute plus an execution price; Replace (U) is the only type carrying *two* order references and shifted shares/price offsets; Trade (P) is an informational print the book treats as a no-op.

Type	Message	Bytes	Book effect
A	Add Order	36	insert, update level
F	Add Order with MPID	40	insert (MPID ignored)
X	Order Cancel	23	reduce shares
D	Order Delete	19	remove order
E	Order Executed	31	reduce shares
C	Order Executed with Price	36	reduce shares
U	Order Replace	35	remove original, insert new
P	Trade (non-cross)	44	none (informational)

9.2 Depth-weighted strategy and risk controls

The strategy now consumes the book’s top 4 levels rather than the touch alone. It forms a **depth-weighted volume** per side — level i weighted ($NLEVELS - i$), so the touch dominates but a supported book still counts — and applies the same fixed-point cross-multiply on those weighted volumes. Three controls were added: a **spread guard** (no trade unless ask > bid and the spread is within `MAX_SPREAD_TICKS` = \$0.10); a **cooldown** (`COOLDOWN_CYCLES` after a fire, preventing order spam); and **imbalance-scaled lot sizing** (from `BASE_LOT`=100 up to `MAX_LOT`=250 shares as imbalance strengthens past $2\times/3\times$ the threshold).

9.3 AXI-Lite latency readout

`latency_counter` exposes its 64-bucket histogram and last-latency value to the host over a 32-bit AXI-Lite slave: histogram buckets at `0x000-0x0FC`, last measured latency at `0x100`, and a clear strobe at `0x104`. The deterministic pipeline produces a single-spike histogram at 39 cycles.

10 What’s Next

Part 2 covered the RTL and its verification. The pipeline is functionally complete and fully tested in simulation. Remaining work, in priority order:

Done since the first M2 cut: `order_book_m2` is wired into `tick_to_trade_top` (195 ns preserved); the parser decodes 8 ITCH types (A, F, C, D, E, U, X, P); the book is multi-level (top 4) with `msg_ready` back-pressure and selective rescan; the strategy is depth-weighted with a spread guard, cooldown and imbalance-scaled lot sizing; and `latency_counter` exposes its histogram over AXI-Lite.

Remaining work, in priority order:

1. **Vivado synthesis / timing closure** on the VU9P part — converts the 195 ns from “simulation cycles $\times$ clock period” into a timing-closed silicon result (run on a larger-RAM machine; see Part 3).
2. **PCIe DMA read path** — so the host reads the 72-bit decision bus from hardware rather than a simulation queue.
3. **Further ITCH coverage** — System Event / NOII messages, and an incremental (rescan-free) multi-level update for sustained burst throughput.